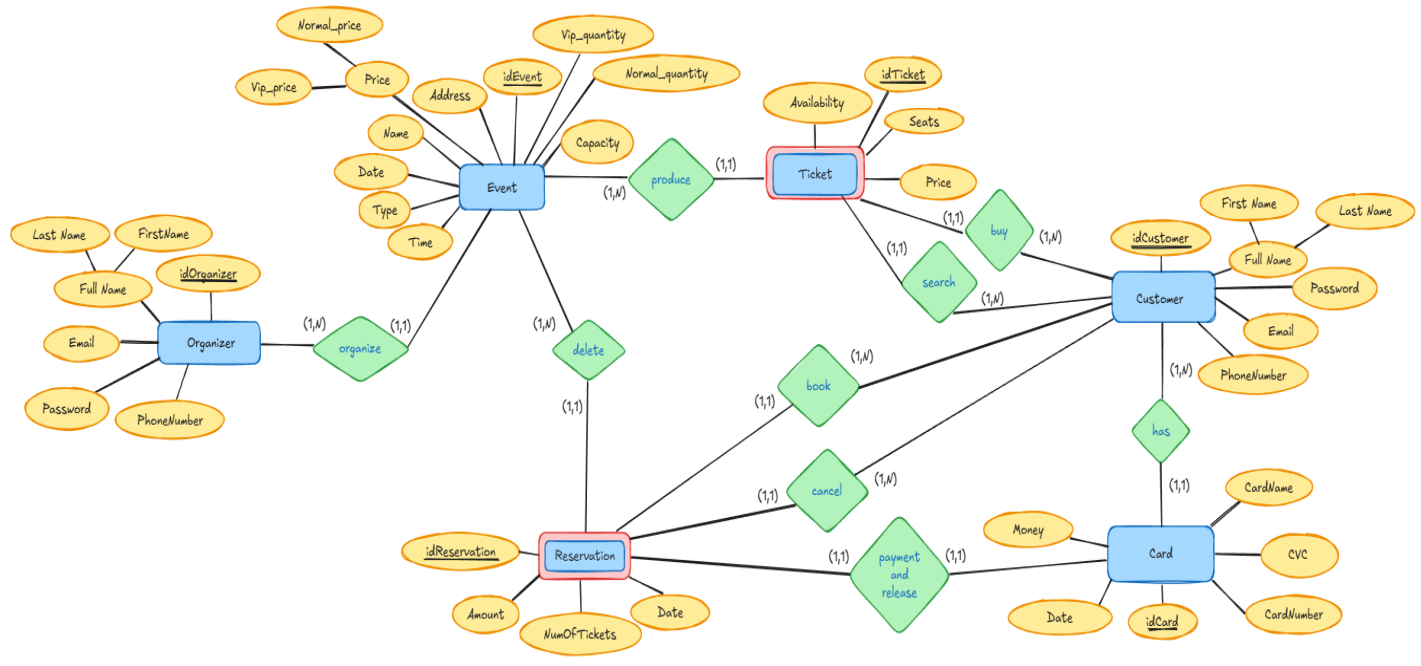




Α' ΦΑΣΗ :

Ένα πλήρες διάγραμμα οντοτήτων-σχέσεων:



Τα γνωρίσματα (όνομα, τύπος) όλων των οντοτήτων και σχέσεων & τα πρωτεύοντα κλειδιά:

Event	varchar(50)	varchar(50)	varchar(50)	date	time	int	int	int	int	int	
idEvent	Name	Address	Type	Date	Time	Normal_price	Normal_quantity	Vip_price	Vip_quantity	Capacity	idOrganizer
Ticket	int	float	tinyint								
idTicket	Seats	Price	Availability	idEvent	idCustomer						
Reservation	date	int	int								
idReservation	Date	Amount	NumOfTickets	idCustomers	idEvent	idCard					
Customer	varchar (50)	varchar (50)	big int	varchar (50)	varchar (50)						
idCustomer	First Name	Last Name	PhoneNumber	Email	Password						
Card	varchar (50)	varchar (16)	date	int	varchar (3)						
idCard	CardName	Card number	Date	Money	CVC	idCustomer					
Organizer	varchar (50)	varchar (50)	big int	varchar (50)	varchar (50)						
idOrganizer	First Name	Last Name	PhoneNumber	Email	Password						

Επεξηγήσεις για τα μη προφανή γνωρίσματα και τις μη-προφανείς σχέσεις:

Vip_quantity & Normal_quantity: Βάλαμε αυτά ορίσματα για διευκόλυνση στην δημιουργία ενός Event. Έτσι ξέρουμε ποσά Vip και κανονικά εισιτήρια να δημιουργήσει ο organizer.

Money: Στην κανονική ζωή σαφώς και δεν ζητάμε ποσά λεφτά υπάρχουν στην κάρτα ενός χρήστη άλλα για το project το κάναμε έτσι μιας που δεν υπάρχει άλλος τρόπος.

Organize: Έχουμε τους organizers(εμείς δηλαδή) που δημιουργούν ή διαγράφουν events γιαυτό υπάρχει αυτή η σχέση.

Δεν έχουμε τίποτα άλλο μη προφανές.

Περιορισμούς πληθικότητας:

Organizer-organize-Event: Οι Organizers μπορούν να οργανώνουν πολλά Events άλλα κάθε Event ανήκει σε έναν μόνο Organizer.

Event-delete-Reservation: Ένα Event διαγράφει όλα τα Reservation άλλα κάθε Reservation διαγράφεται από ένα μόνο Event

Event-produce-Ticket: Ένα Event παράγει πολλά Tickets άλλα κάθε Ticket ανήκει μόνο σε ένα Event.

Ticket-buy-Customer: Οι Customers μπορούν να αγοράσουν πολλά Tickets άλλα κάθε Ticket έχει μοναδικό Customer.

Ticket-search-Customer: Ένας Customer μπορεί να αναζητήσει όλα τα Ticket άλλα κάθε Ticket είναι μοναδικό.

Customer-has-Card: Οι Customers έχουν πολλές Cards άλλα η κάθε Card ανήκει σε έναν Customer.

Customer-book-Reservation: Μια Reservation κλείνεται από έναν Customer άλλα οι Customers μπορούν να κλείνουν πολλές Reservations.

Customer-cancel-Reservation: Ένα Reservation ακυρώνεται από έναν Customer άλλα οι Customers μπορούν να ακυρώσουν πολλές Reservations.

Reservation-Payment and release-Card: Μια Card πληρώνει ή αποδεσμεύει ένα Reservation και ένα Reservation πληρώνεται η αποδεσμεύεται από μια Card.

Τη μετάφραση του μοντέλου σας στο σχεσιακό μοντέλο:

Από τον ορισμό που ισχύει για σχέσεις 1-N βάλουμε τα ξένα κλειδιά στις οντότητες που είναι 1 για να ισχύει η 1 κανονική μορφή. Στην σχέση 1-1 διαλέξαμε το Id της Card να είναι στο Reservation επειδή κάθε Reservation αναγκαστικά πρέπει να πληρωθεί όποτε σίγουρα θα έχει πληρωθεί από μια κάρτα.

Το Ticket είναι ασθενές οντότητα γιατί αν δεν υπάρχει Event δεν υπάρχουν Tickets.

Το Reservation επίσης είναι ασθενές γιατί αν δεν υπάρχουν Customers δεν μπορούν να υπάρξουν Reservations.

Οι εντολές της γλώσσας ορισμού δεδομένων για τις σχέσεις που προκύπτουν:

Δημιουργεί όλους τους πίνακες Ticket,Reservation,Event,Card,Customer,Organizer και βάλουμε και τους εαυτούς μας ως Organizer γιατί δεν υπάρχει άλλος τρόπος δημιουργίας organizer διότι αν είχαμε βάλει να δημιουργούνται στην ιστοσελίδα δεν θα ξέραμε ποιοι πραγματικά είναι οι organizer και έτσι κρατάμε την ασφάλεια της βάσης. Ο κώδικας παρακάτω καλείτε όταν ξεκινήσει το BackEnd Μας γιαυτό μην κάνετε ανανέωση γιατί θα χάσετε τις έγγραφες σας!

```
CREATE DATABASE IF NOT EXISTS project360;
DROP TABLE IF EXISTS Ticket;
DROP TABLE IF EXISTS Reservation;
DROP TABLE IF EXISTS Event;
DROP TABLE IF EXISTS Card;
DROP TABLE IF EXISTS Customer;
DROP TABLE IF EXISTS Organizer;
CREATE TABLE Customer (
    idCustomer INT AUTO_INCREMENT,
    FirstName VARCHAR(50),
    LastName VARCHAR(50),
    PhoneNumber BIGINT,
    Email VARCHAR(50),
    Password VARCHAR(50),
    PRIMARY KEY (idCustomer),
    UNIQUE (PhoneNumber),
    UNIQUE (Email)
);
CREATE TABLE Organizer (
    idOrganizer INT AUTO_INCREMENT,
    FirstName VARCHAR(50),
    LastName VARCHAR(50),
    PhoneNumber BIGINT,
    Email VARCHAR(50),
    Password VARCHAR(50),
    PRIMARY KEY (idOrganizer),
    UNIQUE (PhoneNumber)
);
CREATE TABLE Event (
    idEvent INT AUTO_INCREMENT,
    Name VARCHAR(50),
    Address VARCHAR(50),
    Type VARCHAR(50),
    Date DATE,
    Time TIME,
    Normal_price INT,
    Normal_quantity INT,
    Vip_price INT,
    Vip_quantity INT,
```

```

Capacity INT CHECK (Capacity >= 0),
idOrganizer INT,
PRIMARY KEY (idEvent),
FOREIGN KEY (idOrganizer) REFERENCES Organizer(idOrganizer),
UNIQUE (Name)
);
CREATE TABLE Card (
idCard INT AUTO_INCREMENT,
CardName VARCHAR(50),
CardNumber VARCHAR(16),
Date DATE,
Money INT CHECK (Money >= 0),
CVC VARCHAR(3),
idCustomer INT,
PRIMARY KEY (idCard),
UNIQUE (CardNumber),
FOREIGN KEY (idCustomer) REFERENCES Customer(idCustomer)
);
CREATE TABLE Reservation (
idReservation INT AUTO_INCREMENT,
Date DATE,
Amount INT CHECK (Amount >= 0),
NumOfTickets INT CHECK (NumOfTickets >= 0),
idCustomer INT,
idEvent INT,
idCard INT,
PRIMARY KEY (idReservation),
FOREIGN KEY (idCustomer) REFERENCES Customer(idCustomer),
FOREIGN KEY (idEvent) REFERENCES Event(idEvent),
FOREIGN KEY (idCard) REFERENCES Card(idCard)
);
CREATE TABLE Ticket (
idTicket INT AUTO_INCREMENT,
Seats VARCHAR(10),
Price FLOAT CHECK (Price >= 0),
Availability BOOLEAN,
idEvent INT,
idCustomer INT,
PRIMARY KEY (idTicket),
FOREIGN KEY (idEvent) REFERENCES Event(idEvent),
FOREIGN KEY (idCustomer) REFERENCES Customer(idCustomer)
);
INSERT INTO Organizer (
FirstName,
LastName,
PhoneNumber,
Email,

```

```

    Password
)
VALUES (
    'Antonios',
    'Kavalieros',
    6985623413,
    'csd5088@csd.uoc.gr',
    'AKava'
);
INSERT INTO Organizer (
    FirstName,
    LastName,
    PhoneNumber,
    Email,
    Password
)
VALUES (
    'Emiraldo',
    'Lamakia',
    6973653309,
    'csd5059@csd.uoc.gr',
    'Ilovelol12@'
);
INSERT INTO Organizer (
    FirstName,
    LastName,
    PhoneNumber,
    Email,
    Password
)
VALUES (
    'Georgia',
    'chrysou',
    6973653310,
    'csd4567@csd.uoc.gr',
    'TsigaraForEver'
);

```

Περιορισμούς ακεραιότητας και συναρτησιακές εξαρτήσεις:

Έχουμε Domain Constraints σαν περιορισμούς ακεραιότητας :

1. Capacity INT CHECK (Capacity >= 0)
2. Money INT CHECK (Money >= 0)
3. Amount INT CHECK (Amount >= 0),
4. NumOfTickets INT CHECK (NumOfTickets >= 0)
5. Price FLOAT CHECK (Price >= 0)

Συναρτησιακές εξαρτήσεις:

idOrganizer → First Name, Last Name, PhoneNumber, Email, Password

idCard, idCustomer → CardName, Card number, Date, Money, CVC

idCustomer → First Name, Last Name, PhoneNumber, Email, Password

idReservation, idCustomer , idEvent → Date, Amount, NumOfTickets

idTicket, idEvent, idCustomer → Seats, Price, Availability

idEvent, idOrganizer → Name, Address, Type, Date, Time, Normal_price, Normal_quantity, Vip_price, Vip_quantity, Capacity

Καθορισμό κλειδιών των σχέσεων βάσει των συναρτησιακών εξαρτήσεων:

idOrganizer → First Name, Last Name, PhoneNumber, Email, Password

To idOrganizer αποτελεί το κλειδί της σχέσης.

idCard, idCustomer → CardName, Card number, Date, Money, CVC

To idCard αποτελεί το κλειδί της σχέσης.

idCustomer → First Name, Last Name, PhoneNumber, Email, Password

To idCustomer αποτελεί το κλειδί της σχέσης.

idReservation, idCustomer , idEvent → Date, Amount, NumOfTickets

To idReservation αποτελεί το κλειδί της σχέσης.

idTicket, idEvent, idCustomer → Seats, Price, Availability

To idTicket αποτελεί το κλειδί της σχέσης.

idEvent, idOrganizer → Name, Address, Type, Date, Time, Normal_price, Normal_quantity, Vip_price, Vip_quantity, Capacity

To idEvent αποτελεί το κλειδί της σχέσης.

Δεν έχουμε άλλα υποψηφία κλειδιά γιατί ξέρουμε τα ήδη υπάρχουν κλειδιά μας ούτε θέλουμε νέα υποψηφία κλειδιά.

Μετατροπή του μοντέλου σε τρίτη κανονική μορφή με διατήρηση των συναρτησιακών εξαρτήσεων και χωρίς απώλεια πληροφορίας:

Για να ισχύει η 3 κανονική μορφή πρέπει να ισχύει αναγκαστικά και 1 κανονική μορφή. Λόγω ότι δεν έχουμε διπλότυπα γνωρίσματα ισχύει η 1 κανονική μορφή. Επίσης λόγω των συναρτησιακών εξαρτήσεων που έχουμε ισχύει ο ορισμός της 3 κανονικής μορφής άρα είναι σε 3 κανονική μορφή.

Περιγραφή των ερωτήσεων προς τη βάση δεδομένων με SQL:

```
SELECT * FROM project360.event WHERE idEvent= ?
```

Διαλέγει όλες τις πλειάδες του Event με idEvent (?)

```
DELETE FROM project360.reservation WHERE idReservation= ?
```

Διαγράφει Reservation οπού idReservation είναι (?)

```
SELECT * FROM project360.ticket where idEvent= ? and idCustomer= ?
```

Παίρνει όλα τα Tickets(όλες τις πλειάδες) από συγκρινόμενου idEvent (?) που πήρε ο idCustomer (?)

```
select * from project360.reservation where idCustomer= ?
```

Επιστρέφει όλα τα Reservation (όλες τις πλειάδες) του που έχει πάρει ο idCustomer (?)

```
select * from project360.reservation where idCustomer= ? and Date >= ? and Date <= ?
```

Επιστρέφει όλα τα Reservation (όλες τις πλειάδες) που έχει πάρει ο idCustomer (?) όπου το Date είναι ανάμεσα σε 2 ημερομηνίες (?)

```
SELECT * FROM event
```

Επιστρέφει όλα τα Event (όλες τις πλειάδες)

```
SELECT e.Name AS EventName,SUM(r.Amount) AS TotalRevenue FROM project360.event e JOIN  
project360.reservation r ON e.idEvent = r.idEvent WHERE r.Date BETWEEN ? AND ? GROUP  
BY e.Name ORDER BY TotalRevenue DESC LIMIT 1
```

Επιστρέφει το Name ενός Event ως EventName και το συνολικό άθροισμα από Reservation ως TotalRevenue αφού τα κάνει join, και το date των Reservation είναι ανάμεσα σε (?) και (?) και γίνεται σε φθίνουσα σειρά και επιστρέφει το πρώτο event άρα το event με τα πιο πολλά Reservation


```
INSERT INTO project360.customer (FirstName, LastName, PhoneNumber, Email, Password)
VALUES(?, ?, ?, ?, ?)
```

Προσθέτει ένα Customer στους Customers με values (?)

```
SELECT * FROM project360.customer WHERE phoneNumber = ? or email = ?
```

Διαλέγει τις πλειάδες ενός Customer με phoneNumber (?) ή email (?)

```
SELECT IdCustomer FROM customer WHERE email = ? AND password = ?
```

Διαλέγει τον idCustomer με email (?) και password (?)

```
SELECT COUNT(*) as count FROM project360.ticket WHERE idEvent = ? and Seats= ? and
Availability= ?
```

Επιστρέφει τον μετρητή των Tickets ως count όπου το idEvent είναι (?) το Seats είναι (?) και το Availability είναι (?)

```
SELECT * FROM project360.ticket WHERE Availability= ? and IdEvent= ? and Seats= ?
```

Επιστρέφει όλα τα Tickets (όλες τις πλειάδες) όπου Availability είναι (?), idEvent (?) και Seats είναι (?)

```
select * from project360.event where idEvent=(SELECT idEvent FROM project360.ticket
Where Availability= ? GROUP BY idEvent ORDER BY COUNT(*) DESC LIMIT 1)
```

Διαλέγει όλες τις πλειάδες ενός idEvent όπου αυτό το idEvent έχει τα περισσότερα ticket όπου ισχύει το Availability (?). Γίνεται σε φθίνουσα σειρά και επιστρέφει το πρώτο στοιχείο άρα με το μεγαλύτερο counter από Tickets

```
UPDATE project360.ticket SET Availability = ?, idCustomer = ? WHERE idTicket = ?
```

Ενημερώνει ένα ticket με Availability (?), idCustomer (?) όπου ισχύει το IdTicket (?)

```
SELECT IdOrganizer FROM organizer WHERE email = ? AND password = ?
```

Διαλέγει ένα idOrganizer όπου ισχύει το email (?) και κωδικός (?)

```
INSERT INTO ticket(Seats,Price,Availability,idEvent,idCustomer) VALUES(?,?,?,?,,?)
```

Προσθέτει ένα Ticket με τα values (?)

```
SELECT idEvent FROM event WHERE Name= ?
```

Διαλέγει ένα idEvent όπου το Name είναι (?)

```
Select Money from Card where idCard= ?
```

Διαλέγει τα Money μιας idCard (?)

```
UPDATE Card SET Money = ? WHERE idCard= ?
```

Ενημερώνει το Money με νέα τιμή (?) όπου το idCard είναι (?)

```
INSERT INTO reservation(Date,Amount,NumOfTickets,idCustomer,idEvent,idCard)  
VALUES(?,?,?,?,?,?)
```

Προσθέτει ένα νέο Reservation με values(?)

```
SELECT * FROM project360.card WHERE idCustomer= ?
```

Επιστρέφει όλες τις Card(και τις πλειάδες του) όπου το idCustomer είναι (?)

```
INSERT INTO card (CardName, CardNumber, Date, Money, CVC, idCustomer) VALUES (?, ?, ?,  
?, ?, ?)
```

Προσθέτει μια νέα Card με τα values (?)

```
INSERT INTO event (name, address, type, date, time, normal_price, normal_quantity,  
vip_price, vip_quantity, capacity, IdOrganizer)  
VALUES(?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
```

Προσθέτει ένα νέο Event με τα values (?)

```
SELECT COUNT(*) AS totalEvents FROM project360.event WHERE idOrganizer= ?
```

Επιστρέφει μετρητή των Events ως totalEvents όπου το idOrganizer είναι (?)

```
SELECT * FROM project360.event WHERE idOrganizer= ?
```

Επιστρέφει όλα τα Events (και όλες τις πλειάδες του) όπου το idOrganizer είναι (?)

```
SELECT SUM(Amount) AS totalAmount FROM project360.reservation WHERE idEvent = ?
```

Επιστρέφει το άθροισμα των Amount ως totalAmount όπου το idEvent είναι (?)

```
DELETE FROM project360.ticket WHERE idEvent= ?
```

Διαγράφει ένα Ticket όπου το idEvent είναι (?)

```
SELECT * FROM project360.reservation WHERE idEvent = ?
```

Επιστρέφει ένα Reservation(όλες τις πλειάδες) οπού το idEvent (?)

```
SELECT COUNT(*) AS totalReservations FROM project360.reservation WHERE idEvent= ?
```

Επιστρέφει έναν μετρητή των Reservations σαν totalReservations οπού το idEvent είναι (?)

```
DELETE FROM project360.event WHERE idEvent= ?
```

Διαγράφει ένα event με idEvent (?)

```
SELECT SUM(Price) AS TotalVipBalance FROM project360.ticket WHERE idEvent = ? and  
Availability = ? and Seats = ?
```

Επιστρέφει το άθροισμα των price των ticket σαν TotalVipBalance οπού το idEvent (?) και Availability (?) και Seats (?)

```
SELECT Count(*) AS TotalVipSeats FROM project360.ticket WHERE idEvent = ? and  
Availability = ? and Seats = ?
```

Επιστρέφει το άθροισμα από tickets σαν totalVipSeats οπού το idEvent είναι (?) και το Availability (?) και Seats (?)

Περιγραφή των όψεων (εφόσον χρησιμοποιηθούν):

Δεν χρησιμοποιήσαμε.

Περιγραφή σε ψευδοκώδικα των διαδικασιών:

Δεν έχουμε ψευδοκώδικα πάρα μόνο κωδικά που είναι στην Β' ΦΑΣΗ.

B' ΦΑΣΗ:

Τελική μορφή εννοιολογικού μοντέλου (συνοπτική παρουσίαση και αιτιολόγηση πιθανών αλλαγών σε σχέση με αυτό της φάσης I):

Όλες οι αλλαγές μας γίνανε στην Α' ΦΑΣΗ. Δεν υπάρχουν επιπλέον αλλαγές.

Ενδεικτικά αποτελέσματα από την εκτέλεση των διαδικασιών και ένα εγχειρίδιο χρήσης της εφαρμογής:

README SETUP WARNING:

Για την σωστή λειτουργία του project μας θα πρέπει να εγκαταστήσετε το node.js, συγκεκριμένα την έκδοση v22.11.0 (όχι μικρότερη τουλάχιστον !).

Στη συνέχεια θα πρέπει να εγκαταστήσετε την mySQL. Εμείς δουλεύουμε με την MySQL Workbench 8.0 CE

Έπειτα θα πρέπει να ανοίξετε το project μας μέσω Visual Studio Code και μέσα στον φάκελο Backend στο αρχείο .env να τοποθετήσετε το κωδικό σας στο πεδίο BD_PASSWORD=(αυτόν που βάλατε-έχετε στην βάση σας στην MySQL).

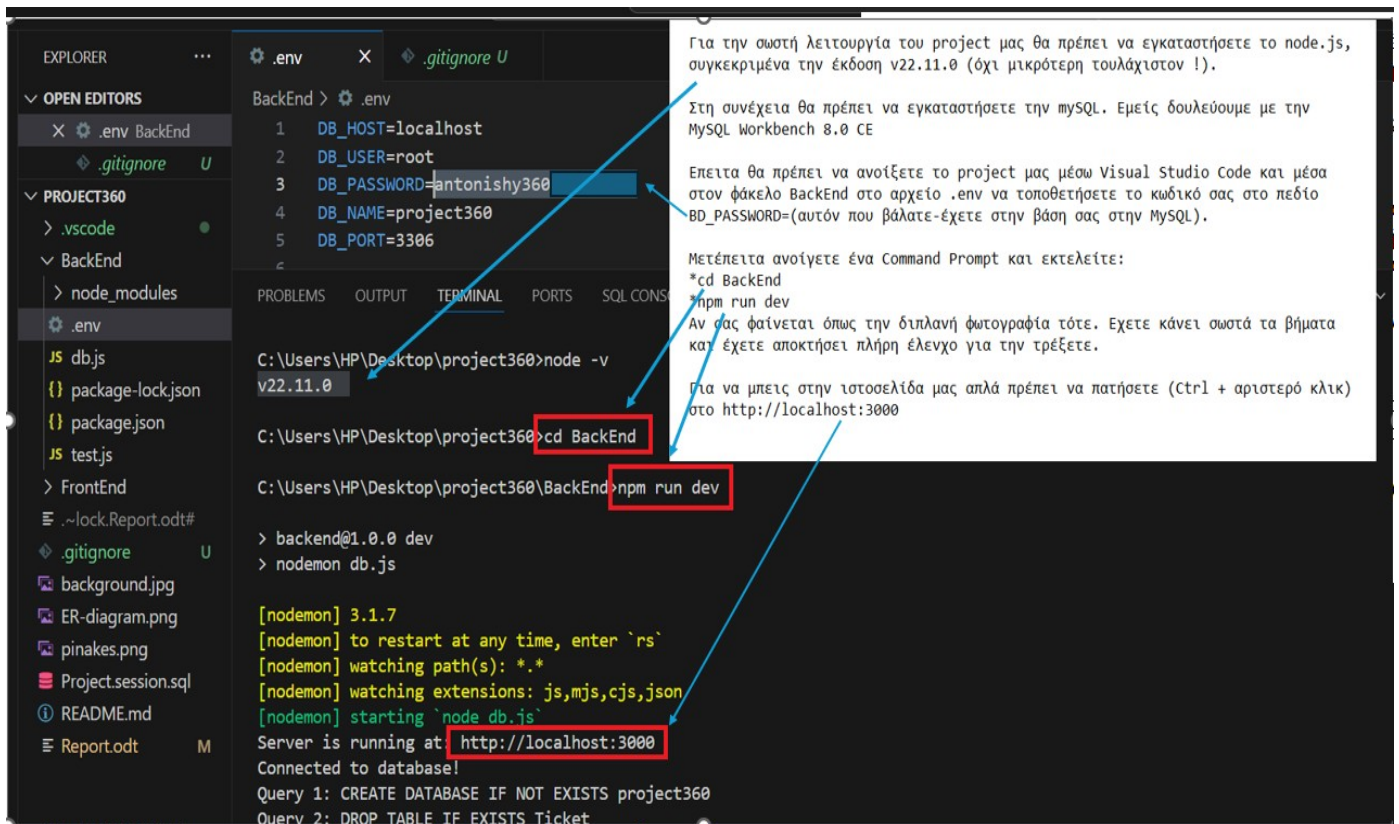
Μετάπειτα ανοίγετε ένα Command Prompt και εκτελείτε:

```
*cd Backend
```

```
*npm run dev
```

Αν σας φαίνεται όπως την φωτογραφία από κάτω τότε. Έχετε κάνει σωστά τα βήματα και έχετε αποκτήσει πλήρη έλεγχο για να το τρέξετε.

Για να μπείτε στην ιστοσελίδα μας απλά πρέπει να πατήσετε (Ctrl + αριστερό κλικ) στο <http://localhost:3000>.



1)

Result Grid Filter Rows: Edit:    Export/Import:  						
	idCustomer	FirstName	LastName	PhoneNumber	Email	Password
▶	1	Antonis	Kavalieros	1234567890	csd5088@csd.uoc.gr	AKava1@
•	NULL	NULL	NULL	NULL	NULL	NULL

2)

Email:

csd5088@csd.uoc.gr

Password:

AKava

☐ Simple user

☒ Organizer

Log In

Sign in

Ο ιστότοπος localhost:3000 λείει

Success Log In by Organizer

OK

3)Αφού πατήσατε Log In θα σας φέρουμε στην page μας όπου γίνονται οι συνδέσεις για τους Simple user(Customers) και για τους Organizers.

4)Τοποθετήστε ένα από τα παρακάτω account. Καθώς μόνο εμείς μπορούμε να είμαστε Organizer και δεν μπορεί το ευρύ κοινό να δημιουργεί ανεξέλεγκτα Organizers.
Email: csd5088@csd.uoc.gr
Password: AKava

Email: csd5059@csd.uoc.gr
Password: Ilovelol12@

Email: csd4567@csd.uoc.gr
Password: TsigaraForEver

5)Πατήστε σύνδεση μέσω Organizer και Log In.
Αν τα βάλατε σωστά θα έχετε το παραπάνω μήνμα

	idOrganizer	FirstName	LastName	PhoneNumber	Email	Password
1	1	Antonios	Kavalieros	6985623413	csd5088@csd.uoc.gr	AKava
2	2	Emiraldo	Lamakia	6973653309	csd5059@csd.uoc.gr	Ilovelol12@
3	3	Georgia	chrysou	6973653310	csd4567@csd.uoc.gr	TsigaraForEver
*	NULL	NULL	NULL	NULL	NULL	NULL

3)

Δημιουργία Εκδήλωσης

Όνομα:

Ολυμπιακός - Παναθηναϊκός

Τοποθεσία:

Γύπεδο Γεώργιος Καραϊσκάκης

Είδος:

Αθλητική

Ημερομηνία:

15/12/2024

Ώρα:

09:00 πμ

Τιμή Κανονικού Εισητηρίου:

5

Τιμή VIP Εισητηρίου:

10

Ποσότητα Κανονικών Εισητηρίων:

10

Ποσότητα VIP Εισητηρίων:

20

Συνολική Ποσότητα Εισητηρίων:

30

Αποθήκευση Εκδήλωσης

Log out

- 6)Σε αυτήν την ιστοσελίδα ο κάθε Orgaizer μπορεί να δει σχεδόν αναλυτικά τα κέρδη του. Το πιο βασικό είναι όμως ότι εδώ δημιουργούνται τα Events. Συμπληρώστε τα πεδία όπως το παράδειγμα (Προσοχή είναι υποχρεωτικό η τιμή των VIP εισητηρίων να είναι μεγαλύτερη από τα κανονικά αλλά αν κάνετε λάθος απλά θα σας βγάλουμε κατάλληλο μήνυμα).
- 7)πατήστε Αποθήκευση εκδήλωσης. θα βγει πάλι κατάλληλο μήνυμα. Πλέον έχουν δημιουργηθεί και όλα τα εισητήρια με Availability=1 (δηλαδή μη κλεισμένα).
- 8)Από εδώ και πέρα μπορείτε να βλέπετε πόσα χρήματα έχει συνολικά το event σας, αλλά και να το ακυρώσετε. Την τελευταία ενέργεια θα την χρησιμοποιήσουμε αργότερα. Καλό θα ήταν να φτιάξετε άλλο ένα event.
- 9)πατήστε Log out

Οι Εκδηλώσεις μου 2

Ολυμπιακός - Παναθηναϊκός 0€ Διαγραφή

ΠΑΟΚ - Αρης 0€ Διαγραφή

idTicket	Seats	Price	Availability	idEvent	idCustomer
5	NORMAL	5	1	1	NULL
6	NORMAL	5	1	1	NULL
7	NORMAL	5	1	1	NULL
8	NORMAL	5	1	1	NULL
9	NORMAL	5	1	1	NULL
10	NORMAL	5	1	1	NULL
11	VIP	10	1	1	NULL
12	VIP	10	1	1	NULL
13	VIP	10	1	1	NULL
14	VIP	10	1	1	NULL
15	VIP	10	1	1	NULL
16	VIP	10	1	1	NULL
17	VIP	10	1	1	NULL
18	VIP	10	1	1	NULL
19	VIP	10	1	1	NULL
20	VIP	10	1	1	NULL
21	VIP	10	1	1	NULL
22	VIP	10	1	1	NULL
23	VIP	10	1	1	NULL
24	VIP	10	1	1	NULL
25	VIP	10	1	1	NULL
26	VIP	10	1	1	NULL
27	VIP	10	1	1	NULL
28	VIP	10	1	1	NULL
29	VIP	10	1	1	NULL
30	VIP	10	1	1	NULL

idEvent	Name	Address	Type	Date	Time	Normal_price	Normal_quantity	Vip_price	Vip_quantity	Capacity	idOrganizer
1	Ολυμπιακός - Παναθηναϊκός	Γύπεδο Γεώργιος Καραϊσκάκης	Αθλητική	2024-12-15	09:00:00	5	10	10	20	30	1
2	ΠΑΟΚ - Αρης	Τούμπα	Αθλητική	2024-12-15	08:00:00	10	20	20	30	50	1

4)

Κλείστε την Εκδήλωσή σας

Εύκολη κράτηση εκδηλώσεων με μερικά απλά βήματα!

less

12)Πλέον βρίσκεστε στην βασική ιστοσελίδα. Όμως για να γίνουν οι αγορές μας τι χρειαζόμαστε? Μια κάρτα. επομένως πατήστε το κουμπί οι κάρτες μου, Προσθήστε μια κάρτα και εκεί θα μεταβείτε σε μια ιστοσελίδα για δημιουργία κάρτας.

13)Βάλτε σωστά στοιχεία όπως card number μόνο 16φιος αριθμός και CVC μόνο 3φιος αριθμός. και να μην υπάρχει αυτή η κάρτα στην βάση.

14)Add Card

15)Βρείτε και πατήστε στο όνομα της κάρτας Πλέον θα μπορείτε να κάνετε αγορές.

16)Επιλέξτε μια εκδήλωση και πατήστε κλείστε εκδήλωση.

ηη/μμ/εεεε

Τα reservations σου

Το πιο κερδοφόρο event

ΠΑΟΚ - Αρης

Ημερομηνία: 15-12-2024

Ώρα: 08:00

Τοποθεσία: Τούμπα

Κλείστε Εκδήλωση

Log out

Οι Κάρτες μου 0€

Οι Κρατήσεις Μου

Πρόσθεσε μια Κάρτα

Card name:

AntonisCreditCard

Card number:

1234567890123456

Date of birth:

20/12/2024

CVC:

123

Amount Available:

1000

Add Card

Πίσω στην αρχική

Οι Κάρτες μου 0€

AntonisCreditCard

Πρόσθεσε μια Κάρτα

Οι Κάρτες μου 1000€

idCard	CardName	CardNumber	Date	Money	CVC	idCustomer
1	AntonisCreditCard	1234567890123456	2024-12-20	1000	123	1

5)

Log out

Κλείστε εισιτήρια για την εκδήλωση: Ολυμπιακός - Παναθηναϊκός

Διαθεσιμότητα: 10 Κανονικές Θέσεις 20 Vip Θέσεις

Τύποι Εισιτηρίων
Κανονικό -5€
Vip - 10€

Τιμή εισιτηρίων: 100€

Πόσα κανονικά Εισιτήρια Θέλεις:

4

Πόσα Vip Εισιτήρια Θέλεις:

8

Ημερομηνία: 15-12-2024

Ωρα: 09:00

Τοποθεσία: Γύπεδο Γεώργιος Καραϊσκάκης

Είδος: Αθλητική

Κλείστε Εισιτήρια

Ολυμπιακός - Παναθηναϊκός

Εισιτήρια: 12

Οι Κάρτες μου 800 €

Οι Κρατήσεις Μου

Διαγραφή

17) Στην συνέχεια θα βγει το παρακάτω πεδίο. Βάλτε όσα εισιτήρια θέλετε. Ακόμα και ποσό που δεν μπορείτε να αγοράσετε με βάση τα χρήματα σας. Τότε θα σας βγάλουμε κατάλληλο μήνυμα και δεν θα αγοραστούν, αλλιώς όταν θα πατηθεί το κλείστε τα εισιτήρια. Θα δημιουργηθεί ένα Reservation.

18) Θα ανανεωθεί και το οι κρατήσεις μου και πατώντας το Διαγραφή θα επιστραφούν τα χρήματα σας και θα ανανεωθούν τα εισιτήρια ως μη κλεισμένα.

19) Πλέον έχουμε 1000-100=900 ευρώ και μια κράτηση με 12 εισιτήρια.

Το Πιο Δημοφιλές Event

Ολυμπιακός - Παναθηναϊκός

20) Επίσης πλέον όλοι οι customers βλέπουν τα νέα διαθέσιμα εισιτήρια.

Κλείστε εισιτήρια για την εκδήλωση: Ολυμπιακός - Παναθηναϊκός

Διαθεσιμότητα: 6 Κανονικές Θέσεις 12 Vip Θέσεις

idTicket	Seats	Price	Availability	idEvent	idCustomer
1	NORMAL	5	0	1	1
2	NORMAL	5	0	1	1
3	NORMAL	5	0	1	1
4	NORMAL	5	0	1	1
5	NORMAL	5	1	1	1
6	NORMAL	5	1	1	1
7	NORMAL	5	1	1	1
8	NORMAL	5	1	1	1
9	NORMAL	5	1	1	1
10	NORMAL	5	1	1	1
11	VIP	10	0	1	1
12	VIP	10	0	1	1
13	VIP	10	0	1	1
14	VIP	10	0	1	1
15	VIP	10	0	1	1
16	VIP	10	0	1	1
17	VIP	10	0	1	1
18	VIP	10	0	1	1

idReservation	Date	Amount	NumOfTickets	idCustomer	idEvent	idCard
1	2024-12-14	100	12	1	1	1

21) Κάνοντας log out και τα ίδια βήματα από το 10) και φτιάχνουμε άλλον έναν customer, κάρτα και κράτηση. Ας υποθέσουμε ότι εκείνος κάνει μια κράτηση στο ΠΑΟΚ - Αρης όπως την παρακάτω φωτογραφία.

6)

Firstname: Admin

Lastname: Oprvtos

Password: 1234567890

Confirm Password: *****

Password is strong

Email: Oprvtos@csl.uoc.gr

Phone: 1234567891

☒ I agree to the terms of use

Card name: AdminCard

Card number: 1234567890123457

Date of birth: 16/12/2024

CVC: 321

Amount Available: 5000

Add Card

Πίσω στην αρχή

Κλείστε εισιτήρια για την εκδήλωση: ΠΑΟΚ - Αρης

Διαθεσιμότητα: 20 Κανονικές Θέσεις 30 Vip Θέσεις

Τύποι Εισιτηρίων
Κανονικό -10€
Vip - 20€

Τιμή εισιτηρίων: 130€

Πόσα κανονικά Εισιτήρια Θέλεις:

13

Πόσα Vip Εισιτήρια Θέλεις:

0

Ημερομηνία: 15-12-2024

Ωρα: 08:00

Τοποθεσία: Τσίμπα

ΠΑΟΚ - Αρης

Οι Κάρτες μου 4870 €

Οι Κρατήσεις Μου

ΠΑΟΚ - Αρης - Εισιτήρια: 13

Διαγραφή

Το Πιο Δημοφιλές Event

Ολυμπιακός - Παναθηναϊκός

Το Πιο Δημοφιλές Event

ΠΑΟΚ - Αρης

idTicket	Seats	Price	Availability	idEvent	idCustomer
30	VIP	10	1	1	1
31	NORMAL	10	0	2	2
32	NORMAL	10	0	2	2
33	NORMAL	10	0	2	2
34	NORMAL	10	0	2	2
35	NORMAL	10	0	2	2
36	NORMAL	10	0	2	2
37	NORMAL	10	0	2	2
38	NORMAL	10	0	2	2
39	NORMAL	10	0	2	2
40	NORMAL	10	0	2	2
41	NORMAL	10	0	2	2
42	NORMAL	10	0	2	2
43	NORMAL	10	0	2	2
44	NORMAL	10	1	2	2
45	NORMAL	10	1	2	2
46	NORMAL	10	1	2	2

22) Ο ΠΑΟΚ - Αρης πλέον έχει 12 κρατήσεις έναντι 11 του Ολυμπιακού - Παναθηναϊκού και για αυτό το ΠΑΟΚ - Αρης γίνεται δημοφιλέστερο και τυχαίνει να είναι και το πιο κερδοφόρο. Αφού έχει 130 έναντι 100.

idCustomer	First Name	Last Name	Phone Number	Email	Password
1	Antonis	Kavalleros	1234567890	csl5088@csl.uoc.gr	AKava1@
2	Admin	Oprvtos	1234567891	Oprvtos@csl.uoc.gr	1234567890

7)

Αναζήτησε εδώ

Από: 14/12/2024

Μέχρι: 16/12/2024

Ta reservations σου

To πιο κερδοφόρο event

Event Ονομα : ΠΑΟΚ - Αρης

Συνολικά κέρδοι : 130

Το Πιο Δημοφιλές Event

Ολυμπιακός - Παναθηναϊκός

25)Αλλάζει και το πιο δημοφιλές αφού πλέον έχουν 16 κρατήσεις ο ΟΛ-ΠΑ και 13 ο ΠΑΟΚ-Αρης και το πιο κερδοφόρο αφού ο πρώτος έχει 140 έναντι 130.

Αναζήτησε εδώ

Από: 14/12/2024

Μέχρι: 16/12/2024

Ta reservations σου

To πιο κερδοφόρο event

Event Ονομα : ΠΑΟΚ - Αρης

Ημερομηνία κράτησης : Sat Dec 14 2024 00:00:00 GMT+0200 (Χαμερινή ώρα Ανατολικής Ευρώπης)

Εισητήρια που πηρες : 13

Event Ονομα : Ολυμπιακός - Παναθηναϊκός

Ημερομηνία κράτησης : Sat Dec 14 2024 00:00:00 GMT+0200 (Χαμερινή ώρα Ανατολικής Ευρώπης)

23)Πάμε να κάνουμε άλλη μια κράτηση από τον ίδιο customer στην παρακάτω εκδήλωση

Κλείστε εισιτήρια για την εκδήλωση: Ολυμπιακός

Διαθεσιμότητα: 6 Κανονικές Θέσεις 12 Vip Θέσεις

Τύποι Εισητηρίων

Κανονικο - 5€

Vip - 10€

Τιμή εισιτηρίων: 40€

Πόσα κανονικά Εισητήρια Θέλεις;

0

Πόσα Vip Εισητήρια Θέλεις;

4

24

14/12/2024

Μέχρι: 16/12/2024

Ta reservations σου

To πιο κερδοφόρο event

Event Ονομα : Ολυμπιακός - Παναθηναϊκός

Συνολικά κέρδοι : 140

26)Τώρα ας γυρίσουμε στο page του Organizer για να δούμε πως φαίνεται και σε εκκίνηση μετά τις αγορές που έχουν γίνει. Log out και βήματα 3 και 4. Απλά βάλτε τον Organizer που τα δημιούργησε καθώς οι άλλοι δεν θα δουν τις αλλαγές παρά μόνο αυτός.

Οι Κάρτες μου 4030 €

Οι Κρατήσεις Μου

ΠΑΟΚ - Αρης - Εισητήρια: 13

Διαγραφή

Ολυμπιακός - Παναθηναϊκός - Εισητήρια: 4

Διαγραφή

idTicket	Seats	Price	Availability	idEvent	idCustomer
1	NORMAL	5	0	1	1
2	NORMAL	5	0	1	1
3	NORMAL	5	0	1	1
4	NORMAL	5	0	1	1
5	NORMAL	5	1	1	1
6	NORMAL	5	1	1	1
7	NORMAL	5	1	1	1
8	NORMAL	5	1	1	1
9	NORMAL	5	1	1	1
10	NORMAL	5	1	1	1
11	VIP	10	0	1	1
12	VIP	10	0	1	1
13	VIP	10	0	1	1
14	VIP	10	0	1	1
15	VIP	10	0	1	1
16	VIP	10	0	1	1
17	VIP	10	0	1	1
18	VIP	10	0	1	1
19	VIP	10	0	1	2
20	VIP	10	0	1	2
21	VIP	10	0	1	2
22	VIP	10	0	1	2
23	VIP	10	1	1	1
24	VIP	10	1	1	1

idReservation	Date	Amount	NumOfTickets	idCustomer	idEvent	idCard
1	2024-12-14	100	12	1	1	1
3	2024-12-14	130	13	2	2	2
4	2024-12-14	40	4	2	1	2

2 και 3 θα πήγαμε αλλά έσβησα ένα γιατί ήθελα να βγαίνει μια συγκεκριμένη περίπτωση

8)

Αναζήτησε εδώ

Από: 14/12/2024

Μέχρι: 16/12/2024

Ta reservations σου To πιο κερδοφόρο event

Event Ονομα : ΠΑΟΚ - Αρης

Συνολικά κέρδοι : 130

Το Πιο Δημοφιλές Event

Ολυμπιακός - Παναθηναϊκός

25) Αλλάζει και το πιο δημοφιλές αφού πλέον έχουν 16 κρατήσεις ο Ολ-Πα και 13 ο Παοκ-Αρης και το πιο κερδοφόρο αφού ο πρώτος έχει 140 έναντι 130.

Αναζήτησε εδώ

Από: 14/12/2024

Μέχρι: 16/12/2024

Ta reservations σου To πιο κερδοφόρο event

Event Ονομα : ΠΑΟΚ - Αρης

Ημερομηνία κράτησης : Sat Dec 14 2024 00:00:00 GMT+0200 (Χαμηρινή ώρα Ανατολικής Ευρώπης)

Εισητήρια που πηρες : 13

Event Ονομα : Ολυμπιακός - Παναθηναϊκός

Ημερομηνία κράτησης : Sat Dec 14 2024 00:00:00 GMT+0200 (Χαμηρινή ώρα Ανατολικής Ευρώπης)

23) Πάμε να κάνουμε άλλη μια κράτηση από τον ίδιο customer στην παρακάτω εκδήλωση

Κλείστε εισιτήρια για την εκδήλωση: Ολυμπιακός

Διαθεσιμότητα: 6 Κανονικές Θέσεις 12 Vip Θέσεις

Τύποι Εισητηρίων

Κανονικο - 5€

Vip - 10€

Τιμή εισιτηρίων: 40€

Πόσα κανονικά Εισητήρια Θέλει;

0

Πόσα Vip Εισητήρια Θέλει;

4

24

14/12/2024

Μέχρι:

16/12/2024

Ta reservations σου To πιο κερδοφόρο event

Event Ονομα : Ολυμπιακός - Παναθηναϊκός

Συνολικά κέρδοι : 140

idReservation	Date	Amount	NumOfTickets	idCustomer	idEvent	idCard
1	2024-12-14	100	12	1	1	1
3	2024-12-14	130	13	2	2	2
4	2024-12-14	40	4	2	1	2

2 και 3 θα πήγαινε αλλά έσβησα ένα γιατί ήθελα να βγαίνει μια συγκεκριμένη περίπτωση

24) πλέον θα έχει αυτές τις κρατήσεις αλλά μπορεί να δει και γενικά για συγκεκριμένες μέρες.

idTicket	Seats	Price	Availability	idEvent	idCustomer
1	NORMAL	5	0	1	1
2	NORMAL	5	0	1	1
3	NORMAL	5	0	1	1
4	NORMAL	5	0	1	1
5	NORMAL	5	1	1	1
6	NORMAL	5	1	1	1
7	NORMAL	5	1	1	1
8	NORMAL	5	1	1	1
9	NORMAL	5	1	1	1
10	NORMAL	5	1	1	1
11	VIP	10	0	1	1
12	VIP	10	0	1	1
13	VIP	10	0	1	1
14	VIP	10	0	1	1
15	VIP	10	0	1	1
16	VIP	10	0	1	1
17	VIP	10	0	1	1
18	VIP	10	0	1	1
19	VIP	10	0	1	2
20	VIP	10	0	1	2
21	VIP	10	0	1	2
22	VIP	10	0	1	2
23	VIP	10	1	1	1
24	VIP	10	1	1	1

26) Τώρα ας γυρίσουμε στο page του organizer για να δούμε πως φαίνεται και σε εκείνον μετά τις αγορές που έχουν γίνει. Log out και βήματα 3 και 4. Αλλά βάλε τον organizer που τα δημιουργήσαμε καθώς οι άλλοι δεν θα δουν τις αλλαγές παρά μόνο αυτός.

9)

Τα συνολικά σου έσοδα είναι.

270€

Γενικά έσοδα από VIP θέσεις 120€

Γενικά έσοδα από κανονικές θέσεις 150€



27) Του βγάξει τα συνολικά του έσοδα και πόσες θέσεις έχει κλπ καθώς και για κάθε event του.

Ο ιστότοπος localhost:3000 λέει

Event, reservations, and all tickets deleted successfully. All clients have received their money back !!!

Οι Εκδηλώσεις μου 1

ΠΑΟΚ - Αρης 130€ Διαγραφή

OK

Τα συνολικά σου έσοδα είναι.

130€

28) Ας πατήσουμε να διαγράψουμε την εκδήλωση Ολυμπιακός-Παναθηναϊκός

29) Έχουν διαγραφεί τα εισιτήρια, οι κρατήσεις. Λέει τα κανονικά ποσά και έχει επιστρέψει τα χρήματα στους κατόχους.

έσοδα από VIP θέσεις

0€

Γενικά έσοδα από καν

130€

Κλείστε εισιτήρια για την εκδήλωση: ΠΑΟΚ - Αρης

Διαθεσιμότητα: 20 Κανονικές Θέσεις 30 Vip Θέσεις

Ο ιστότοπος localhost:3000 λέει

Deleted reservation successfully and all tickets you bought

Οι Κάρτες μου 5000€

Οι Κρατήσεις Μου

Οι Κάρτες μου 4870€

Οι Κρατήσεις Μου

ΠΑΟΚ - Αρης - Εισητήρια: 13 Διαγραφή

idTicket	Seats	Price	Availability	idEvent	idCustomer
31	NORMAL	10	0	2	2
32	NORMAL	10	0	2	2
33	NORMAL	10	0	2	2
34	NORMAL	10	0	2	2
35	NORMAL	10	0	2	2
36	NORMAL	10	0	2	2
37	NORMAL	10	0	2	2
38	NORMAL	10	0	2	2
39	NORMAL	10	0	2	2
40	NORMAL	10	0	2	2
41	NORMAL	10	0	2	2
42	NORMAL	10	0	2	2
43	NORMAL	10	0	2	2
44	NORMAL	10	1	2	1
45	NORMAL	10	1	2	1
46	NORMAL	10	1	2	1
47	NORMAL	10	1	2	1
48	NORMAL	10	1	2	1

VIP Θέσεις Κανονικές Θέσεις

idReservation	Date	Amount	NumOfTickets	idCustomer	idEvent	idCard
3	2024-12-14	130	13	2	2	2

idCard	CardName	CardNumber	Date	Money	CVC	idCustomer
1	AntonisCreditCard	1234567890123456	2024-12-20	1000	123	1
2	AdminCard	1234567890123457	2024-12-16	4870	321	2

Οι Κάρτες μου 1000€

Οι Κρατήσεις Μου

10)

idReservation	Date	Amount	NumOfTickets	idCustomer	idEvent	idCard
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Οι Εκδηλώσεις μου 1

ΠΑΟΚ - Αρης 08 Διαγραφή

Τα συνολικά σου έσοδα είναι.

0€

31)Μετά από τις όποιες αλλαγές η βάση μας θα είναι κάπως έτσι.

Έσοδα από VIP θέσεις

0€

Γενικά έσοδα από κανονικές θέσεις

0€

idReservation	Date	Amount	NumOfTickets	idCustomer	idEvent	idCard
NULL	NULL	NULL	NULL	NULL	NULL	NULL

idTicket	Seats	Prio	Availability	idEvent	idCustomer
31	NORMAL	10	1	2	NULL
32	NORMAL	10	1	2	NULL
33	NORMAL	10	1	2	NULL
34	NORMAL	10	1	2	NULL
35	NORMAL	10	1	2	NULL
36	NORMAL	10	1	2	NULL
37	NORMAL	10	1	2	NULL
38	NORMAL	10	1	2	NULL
39	NORMAL	10	1	2	NULL
40	NORMAL	10	1	2	NULL
41	NORMAL	10	1	2	NULL
42	NORMAL	10	1	2	NULL
43	NORMAL	10	1	2	NULL
44	NORMAL	10	1	2	NULL
45	NORMAL	10	1	2	NULL
46	NORMAL	10	1	2	NULL

idEvent	Name	Address	Type	Date	Time	Normal_price	Normal_quantity	Vip_price	Vip_quantity	Capacity	idOrganizer
2	ΠΑΟΚ - Αρης	Τούμνα	Αθλητική	2024-12-15	08:00:00	10	20	20	30	50	1

Περιγραφή των περιορισμών της υλοποίησής σας και των δυνατοτήτων βελτίωσής του:

Ένα περιορισμό που έχουμε είναι ότι ο χρήστης δεν μπορεί πότε να δημιουργήσει νέους Organizers γιατί δεν του παρέχουμε αυτήν την δυνατότητα στην υλοποίηση μας. Αυτό το κάναμε για ευκολία πρώτα από όλα άλλα και για ασφάλεια στο σύστημα μας ώστε να μην μπορεί ο καθένας να δημιουργεί Events πάρα μόνο εμείς. Άλλο ένα περιορισμό που έχουμε είναι ότι στα Events θα μπορούσαμε οι Organizers να βάζανε και μια φωτογραφία στα Events τους για πιο ωραία αισθητική αλλά λόγω χρόνου δεν το προσθέσαμε.

τον κώδικα των προγραμμάτων που θα υλοποιούν τις διαδικασίες που καθορίστηκαν στην προηγούμενη φάση:

Επιστρέφει όλες τις πληροφορίες ενός Event.

```
app.get("/eventInfo", (req, res) => {
  const idEvent = req.query.idEvent;
  db.query(
    "SELECT * FROM project360.event WHERE idEvent= ?",
    [idEvent],
    (err, results) => {
      if (err) {
        console.error("Database error:", err);
        return res.status(500).send("Error connecting to the database");
      } else {
        return res.status(200).json(results);
      }
    }
  )
})
```

```
);  
});
```

Διαγράφει ένα Reservation.

```
app.delete("/deleteReservation", (req, res) => {  
  const idReservation = req.query.idReservation;  
  db.query(  
    "DELETE FROM project360.reservation WHERE idReservation= ?",  
    [idReservation],  
    (err, results) => {  
      if (err) {  
        console.error("Database error:", err);  
        return res.status(500).send("Error connecting to the database");  
      } else {  
        return res.status(200).send("Reservation deleted successfully");  
      }  
    }  
  );  
});
```

Επιστρέφει όλα τα Tickets ενός Event που έχει κλείσει ένας Customer.

```
app.get("/getalltickets", (req, res) => {  
  const CustomerId = req.cookies.userId;  
  const idEvent = req.query.idEvent;  
  db.query(  
    "SELECT * FROM project360.ticket where idEvent= ? and idCustomer= ?",  
    [idEvent, CustomerId],  
    (err, results) => {  
      if (err) {  
        console.error("Database error:", err);  
        return res.status(500).send("Error connecting to the database");  
      } else {  
        return res.status(200).json(results);  
      }  
    }  
  );  
});
```

Επιστρέφει όλες τις Reservation ενός Customer.

```

app.get("/getReserves", (req, res) => {
  const CustomerId = req.cookies.userId;
  db.query(
    "select * from project360.reservation where idCustomer= ?", [CustomerId],
    (err, results) => {
      if (err) {
        console.error("Database error:", err);
        return res.status(500).send("Error connecting to the database");
      } else {
        return res.status(200).json(results);
      }
    }
  );
});

```

Επιστρέφει όλες τις κρατήσεις ενός Customer για ένα χρονικό διάστημα.

```

app.get("/getReserve", (req, res) => {
  const CustomerId = req.cookies.userId;
  const {from,until}=req.query;
  db.query(
    "select * from project360.reservation where idCustomer= ? and Date >= ? and Date <= ?",
    [CustomerId,from,until],
    (err, results) => {
      if (err) {
        console.error("Database error:", err);
        return res.status(500).send("Error connecting to the database");
      } else {
        return res.status(200).json(results);
      }
    }
  );
});

```

Επιστρέφει όλα τα Event.

```

app.get("/getevents", (req, res) => {
  db.query("SELECT * FROM event", (err, results) => {
    if (err) {
      console.error("Database error:", err);
      return res.status(500).send("Error connecting to the database");
    } else {
      return res.status(200).json(results);
    }
  });
});

```

```

    }
  });
});

```

Επιστρέφει το ακριβότερο Event.

```

app.get("/MostPriceEvent", (req, res) => {
  const { from, until } = req.query;
  db.query("SELECT e.Name AS EventName, SUM(r.Amount) AS TotalRevenue FROM
project360.event e JOIN project360.reservation r ON e.idEvent = r.idEvent WHERE r.Date BETWEEN
? AND ? GROUP BY e.Name ORDER BY TotalRevenue DESC LIMIT 1",
  [from, until], (err, result) =>
  {
    if (err)
    {
      console.error("Database error:", err);
      return res.status(500).send("Error connecting to the database");
    }
    else
    {
      return res.status(200).json(result);
    }
  });
});

```

Αρχικοποιεί στην βάση έναν νέο Customer.

```

app.post("/signup", (req, res) => {
  const { FirstName, LastName, PhoneNumber, Email, Password } = req.body;
  db.query(
    "INSERT INTO project360.customer (FirstName, LastName, PhoneNumber, Email, Password)
VALUES(?, ?, ?, ?, ?)",
    [FirstName, LastName, PhoneNumber, Email, Password],
    (err, results) => {
      if (err) {
        console.error("Error inserting data:", err);
        return res
          .status(500)
          .send({ status: "error", message: "Internal server error" });
      } else {
        const userId = results.insertId;
        res.cookie("userId", userId, {
          httpOnly: true,
          secure: process.env.NODE_ENV === "production",

```

```

    maxAge: 24 * 60 * 60 * 1000,
  });

  return res.status(200).send({
    status: "success",
    message: "User registered successfully",
    userId: userId,
  });
}
}
);
});

```

Ελέγχει αν υπάρχει το ίδιο τηλέφωνο και email.

```

app.get("/checkuser", (req, res) => {
  const { PhoneNumber, Email } = req.query;
  db.query(
    "SELECT * FROM project360.customer WHERE phoneNumber = ? or email = ?",
    [PhoneNumber, Email],
    (err, results) => {
      if (err) {
        console.error("Error querying data:", err);
        return res
          .status(500)
          .send({ status: "error", message: "Internal server error" });
      }
      else {

        if (results.length > 0)
        {
          return res.status(200).send({
            status: "success",
            message: "User exists",
          });
        }
        else
        {
          return res.status(404).send({
            status: "not_found",
            message: "User not found",
          });
        }
      }
    }
  );
});
});

```

Ελέγχει αν ένας Customer έχει βάλει σωστά το email και password του.

```
app.post("/login", (req, res) => {
  const { email, password } = req.body;
  db.query(
    "SELECT IdCustomer FROM customer WHERE email = ? AND password = ?",
    [email, password],
    (err, results) => {
      if (err) {
        return res.status(500).send("Error connecting to the database");
      }
      if (results.length > 0) {
        const customerId = results[0].IdCustomer;
        res.cookie("userId", customerId, {
          httpOnly: true,
          secure: process.env.NODE_ENV === "production",
          maxAge: 2 * 60 * 60 * 1000,
        });

        return res
          .status(200)
          .send({ status: "success", message: "User exists" });
      } else {
        return res
          .status(409)
          .send({ status: "error", message: "User doesnt exists" });
      }
    }
  );
});
```

Επιστρέφει πόσα διαθέσιμα Tickets υπάρχουν για ένα Event.

```
app.get("/ticketsforEvent", (req, res) => {
  const eventId = req.query.EventId;
  const type = req.query.type;
  db.query(
    "SELECT COUNT(*) as count FROM project360.ticket WHERE idEvent = ? and Seats= ? and Availability= ?",
    [eventId, type, false],
    (err, results) => {
      if (err) {
        console.log(err);
        return res.status(500).send({ message: "error", status: "error" });
      } else {

```



```

    return res.status(200).json(results);
  }
}
);
});

```

Επιστρέφει όλα τα διαθέσιμα Ticket για ένα Event.

```

app.get("/AvailableTicket", (req, res) => {
  const EventId = req.query.EventId;
  const Seats = req.query.Seats;
  db.query(
    "SELECT * FROM project360.ticket WHERE Availability= ? and IdEvent= ? and Seats= ?",
    [true, EventId, Seats],
    (err, results) => {
      if (err) {
        console.log(err);
        return res.status(500).send({ message: "error", status: "error" });
      } else {
        return res.status(200).json(results);
      }
    }
  );
});

```

Επιστρέφει το διασημότερο Event

```

app.get("/Popular", (req, res) => {
  db.query(
    "select * from project360.event where idEvent=(SELECT idEvent FROM project360.ticket Where Availability= ? GROUP BY idEvent ORDER BY COUNT(*) DESC LIMIT 1)",
    [false],
    (err, results) => {
      if (err) {
        console.log(err);
        return res.status(500).send("Error with query");
      } else {
        return res.status(200).json(results);
      }
    }
  );
});

```

Ανανεώνει τα Tickets ως κλεισμένα.

```
app.put("/SetTicket", (req, res) => {
  const { idTicket } = req.body;
  const CustomerId = req.cookies.userId;
  db.query(
    "UPDATE project360.ticket SET Availability = ?, idCustomer = ? WHERE idTicket = ?",
    [false, CustomerId, idTicket],
    (err, results) => {
      if (err) {
        console.log(err);
        return res.status(500).send({ message: "error", status: "error" });
      } else {
        return res
          .status(200)
          .send({ message: "Ticket set successfully", status: "success" });
      }
    }
  );
});
```

Ανανεώνει τα Ticket ως ξανά διαθέσιμα.

```
app.put("/SetTickettrue", (req, res) => {
  const { idTicket } = req.body;
  db.query(
    "UPDATE project360.ticket SET Availability = ?, idCustomer = ? WHERE idTicket = ?",
    [true, null, idTicket],
    (err, results) => {
      if (err) {
        console.log(err);
        return res.status(500).send({ message: "error", status: "error" });
      } else {
        return res
          .status(200)
          .send({ message: "Ticket set successfully", status: "success" });
      }
    }
  );
});
```

Ελέγχει αν ένας Organizer έχει βάλει σωστά το email και password του.

```

app.get("/loginOrganizer", (req, res) => {
  const { email, password } = req.query;
  db.query(
    "SELECT IdOrganizer FROM organizer WHERE email = ? AND password = ?",
    [email, password],
    (err, results) => {
      if (err) {
        return res.status(500).send("Error connecting to the database");
      }
      if (results.length > 0) {
        const OrganizerId = results[0].IdOrganizer;
        res.cookie("organizerId", OrganizerId, {
          httpOnly: true,
          secure: process.env.NODE_ENV === "production",
          maxAge: 2 * 60 * 60 * 1000,
        });
        return res
          .status(200)
          .send({ status: "success", message: "Organizer exists" });
      } else {
        return res
          .status(409)
          .send({ status: "error", message: "Organizer doesn't exist" });
      }
    }
  );
});

```

Δημιουργεί τα Ticket για το Event

```

app.post("/addticket", (req, res) => {
  const { Seats, Price, Availability, idEvent } = req.body;
  db.query(
    "INSERT INTO ticket(Seats,Price,Availability,idEvent,idCustomer) VALUES(?,?,?,?,?)",
    [Seats, Price, Availability, idEvent, null],
    (err, results) => {
      if (err) {
        return res
          .status(400)
          .send({ status: "failed", message: "Couldnt add ticket" });
      } else {
        return res
          .status(200)
          .send({ status: "success", message: "Ticket added successfully" });
      }
    }
  );
});

```

```
    }  
  );  
});
```

Επιστρέφει το idEvent με βάση το όνομα του.

```
app.get("/eventid", (req, res) => {  
  const { EventName } = req.query;  
  db.query(  
    "SELECT idEvent FROM event WHERE Name= ?",  
    [EventName],  
    (err, results) => {  
      if (err) {  
        return res  
          .status(400)  
          .send({ status: "failed", message: "Couldnt find the eventId" });  
      } else {  
        return res.status(200).json(results);  
      }  
    }  
  );  
});
```

Επιστρέφει τα χρήματα μιας κάρτας.

```
app.get("/cardMoney", (req, res) => {  
  const idCard = req.query.idCard;  
  db.query(  
    "Select Money from Card where idCard= ?",  
    [idCard],  
    (err, results) => {  
      if (err) {  
        console.log(err);  
        return res.status(400).send("error");  
      } else {  
        return res.status(200).json(results);  
      }  
    }  
  );  
});
```

Ανανεώνει το ποσό μιας κάρτας.

```

app.put("/money", (req, res) => {
  const { idCard, Money } = req.query;
  db.query(
    "UPDATE Card SET Money = ? WHERE idCard= ?",
    [Money, idCard],
    (err, results) => {
      if (err) {
        console.log(err);
        return res
          .status(400)
          .send({ status: "failed", message: "Couldnt find the eventId" });
      } else {
        return res.status(200).send({ status: "success" });
      }
    }
  );
});

```

Δημιουργεί μια Reservation.

```

app.post("/addreservation", (req, res) => {
  const { Date, Amount, NumOfTickets, idEvent, idCard } = req.body;
  const idCustomer = req.cookies.userId;
  db.query(
    "INSERT INTO reservation(Date,Amount,NumOfTickets,idCustomer,idEvent,idCard)
VALUES(?,?,?,?,?,?)",
    [Date, Amount, NumOfTickets, idCustomer, idEvent, idCard],
    (err, results) => {
      if (err) {
        console.log(err);
        return res
          .status(400)
          .send({ status: "failed", message: "Couldnt add the reservation" });
      } else {
        return res
          .status(200)
          .send({ status: "success", message: "reservation added" });
      }
    }
  );
});

```

Επιστρέφει όλες τις Card ενός Customer.

```

app.get("/getcards", (req, res) => {
  const IdCustomer = req.cookies.userId;
  db.query(
    "SELECT * FROM project360.card WHERE idCustomer= ?",
    [IdCustomer],
    (err, results) => {
      if (err) {
        console.error("Database error:", err);
        return res.status(500).send("Error connecting to the database");
      } else {
        return res.status(200).json(results);
      }
    }
  );
});

```

Δημιουργεί μια Card.

```

app.post("/cardadd", (req, res) => {
  const { CardName, CardNumber, Date, Money, CVC } = req.body;
  const idCustomer = req.cookies.userId;
  if (!idCustomer) {
    return res
      .status(400)
      .send({ status: "error", message: "User ID not found in cookies" });
  }
  db.query(
    "INSERT INTO card (CardName, CardNumber, Date, Money, CVC, idCustomer) VALUES",
    (?, ?, ?, ?, ?, ?)",
    [CardName, CardNumber, Date, Money, CVC, idCustomer],
    (err) => {
      if (err) {
        console.error("Error inserting data:", err);
        return res
          .status(500)
          .send({ status: "error", message: "Internal server error" });
      }
      return res
        .status(200)
        .send({ status: "success", message: "Card added successfully" });
    }
  );
});

```

Δημιουργεί ένα Event.

```
app.post("/addevent", (req, res) => {
  const IdOrganizer = req.cookies.organizerId;
  const {name, address, type, date, time, normal_price, normal_quantity, vip_price, vip_quantity,
capacity,} = req.body;
  db.query(
    "INSERT INTO event (name, address, type, date, time, normal_price, normal_quantity, vip_price,
vip_quantity, capacity, IdOrganizer) VALUES(?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)",
    [name, address, type, date, time, normal_price, normal_quantity, vip_price, vip_quantity, capacity,
IdOrganizer,],
    (err, results) => {
      if (err) {
        console.error("Error inserting data:", err);
        return res
          .status(500)
          .send({ status: "error", message: "Internal server error" });
      } else {
        return res.status(200).send({ status: "success make a event" });
      }
    }
  );
});
```

Επιστρέφει πόσα Event υπάρχουν.

```
app.get('/geteventscount', (req, res) => {

  const IdOrganizer = req.cookies.organizerId;
  db.query('SELECT COUNT(*) AS totalEvents FROM project360.event WHERE idOrganizer= ?',
[IdOrganizer], (err, results) => {
    if (err) {
      console.error('Database error:', err);
      return res.status(500).send('Error connecting to the database');
    }
    else {
      console.log('Query results edp:', results);
      return res.status(200).json(results);
    }
  });
});
```

Επιστρέφει όλα τα Event ενός Organizer.

```

app.get('/getOrganizerevents', (req, res) => {
  const IdOrganizer = req.cookies.organizerId;
  db.query('SELECT * FROM project360.event WHERE idOrganizer= ?', [IdOrganizer], (err, results)
=> {
    if (err) {
      console.error('Database error:', err);
      return res.status(500).send('Error connecting to the database');
    }
    else {
      console.log('Query results edp:', results);
      return res.status(200).json(results);
    }
  });
});

```

Επιστρέφει όλα τα έσοδα από ένα Event.

```

app.get('/getAmountForMenu', (req, res) => {
  const idEvent = req.query.idEvent;
  db.query(
    "SELECT SUM(Amount) AS totalAmount FROM project360.reservation WHERE idEvent = ?",
    [idEvent],
    (err, results) => {
      if (err) {
        console.error("Database error:", err);
        return res.status(500).send("Error connecting to the database");
      } else {
        return res.status(200).json(results);
      }
    }
  );
});

```

Διαγράφει όλα τα Tickets για ένα Event.

```

app.delete('/deletealltickets', (req, res) => {
  const idEvent = req.query.idEvent;
  db.query(
    "DELETE FROM project360.ticket WHERE idEvent= ?",
    [idEvent],
    (err, results) => {
      if (err) {

```



```

    console.error("Database error:", err);
    return res.status(500).send("Error connecting to the database");
  } else {
    return res.status(200).send("Reservation deleted successfully");
  }
}
);
});

```

Επιστρέφει όλες τις Reservation για ένα Event.

```

app.get('/getReservations', (req, res) => {
  const idEvent = req.query.idEvent;
  db.query(
    "SELECT * FROM project360.reservation WHERE idEvent = ?",
    [idEvent],
    (err, results) => {
      if (err) {
        console.error("Database error:", err);
        return res.status(500).send("Error connecting to the database");
      } else {
        return res.status(200).json(results);
      }
    }
  );
});

```

Επιστρέφει πόσες Reservation έχουν γίνει για ένα Event.

```

app.get('/getCountReservations', (req, res) => {
  const idEvent = req.query.idEvent;
  db.query('SELECT COUNT(*) AS totalReservations FROM project360.reservation WHERE
idEvent= ?',
    [idEvent],
    (err, results) => {
      if (err) {
        console.error('Database error:', err);
        return res.status(500).send('Error connecting to the database');
      }
      else {
        console.log('Query results edp:', results);
        return res.status(200).json(results);
      }
    }
  );
});

```

```
});
```

Διαγράφει ένα Event.

```
app.delete('/deleteEvent', (req, res) => {
  const idEvent = req.query.idEvent;
  db.query(
    "DELETE FROM project360.event WHERE idEvent= ?",
    [idEvent],
    (err, results) => {
      if (err) {
        console.error("Database error:", err);
        return res.status(500).send("Error connecting to the database");
      } else {
        return res.status(200).send("Reservation deleted successfully");
      }
    }
  );
});
```

Επιστρέφει το ποσό των VIP θέσεων για ένα Event.

```
app.get('/getVipBalance', (req, res) => {
  const idEvent = req.query.idEvent;
  db.query(
    "SELECT SUM(Price) AS TotalVipBalance FROM project360.ticket WHERE idEvent = ? and Availability = ? and Seats = ?",
    [idEvent, false, "VIP"],
    (err, results) => {
      if (err) {
        console.error("Database error:", err);
        return res.status(500).send("Error connecting to the database");
      } else {
        return res.status(200).json(results);
      }
    }
  );
});
```

Επιστρέφει το ποσό των κανονικών θέσεων για ένα Event.

```
app.get('/getNormalBalance', (req, res) => {
  const idEvent = req.query.idEvent;
```

```

db.query(
  "SELECT SUM(Price) AS TotalNormalBalance FROM project360.ticket WHERE idEvent = ? and
  Availability = ? and Seats = ?",
  [idEvent, false, "NORMAL"],
  (err, results) => {
    if (err) {
      console.error("Database error:", err);
      return res.status(500).send("Error connecting to the database");
    } else {
      return res.status(200).json(results);
    }
  }
);
});

```

Επιστρέφει πόσες VIP θέσεις έχουν κλειστεί για ένα Event.

```

app.get('/getVipSeats', (req, res) => {
  const idEvent = req.query.idEvent;
  db.query(
    "SELECT Count(*) AS TotalVipSeats FROM project360.ticket WHERE idEvent = ? and Availability
    = ? and Seats = ?",
    [idEvent, false, "VIP"],
    (err, results) => {
      if (err) {
        console.error("Database error:", err);
        return res.status(500).send("Error connecting to the database");
      } else {
        return res.status(200).json(results);
      }
    }
  );
});

```

Επιστρέφει πόσες κανονικές θέσεις έχουν κλειστεί για ένα Event.

```

app.get('/getNormalSeats', (req, res) => {
  const idEvent = req.query.idEvent;
  db.query(
    "SELECT Count(*) AS TotalNormalSeats FROM project360.ticket WHERE idEvent = ? and
    Availability = ? and Seats = ?",
    [idEvent, false, "NORMAL"],
    (err, results) => {

```

```
    if (err) {  
      console.error("Database error:", err);  
      return res.status(500).send("Error connecting to the database");  
    } else {  
      return res.status(200).json(results);  
    }  
  }  
);  
});
```